DOKUMENTACJA TECHNICZNA IMPLEMENTACJI FRONTENDU

Moduł Dewelopera D: Autentykacja, Oszczędności, Inwestycje, Kalkulator Emerytalny

Projekt	FinApp - Osobisty Asystent Finansowy	**Data aktualizacji**	27 maja 2026 r.
Autor	Deweloper D (Frontend Lead)	**Status**	Zaakceptowany / Wdrożony do main
Technologia	Flutter (Dart), Riverpod	**Wersja modułu**	v1.1.0-MVP

1. Podsumowanie Menedżerskie (Executive Summary)

Niniejszy dokument stanowi oficjalne podsumowanie prac inżynieryjnych wykonanych przez **Dewelopera D** w ramach implementacji warstwy wizualnej oraz logicznej aplikacji **FinApp**. Zakres prac obejmował pełną realizację czterech kluczowych komponentów funkcjonalnych aplikacji: modułu autentykacji (ekran logowania), modułu kont oszczędnościowych, modułu kont inwestycyjnych oraz interaktywnego kalkulatora emerytalnego.

Architektura rozwiązania została zaprojektowana w oparciu o paradygmat stanowy **Riverpod 2.0**, gwarantując pełną separację logiki biznesowej od interfejsu użytkownika (UI). Rozwiązanie pomyślnie przeszło testy integracyjne z lokalną symulacją API (Mock API) dla JWT oraz CRUD i zostało scalone z gałęzią główną (`main`), stanowiąc stabilną bazę pod rozwój pozostałych składowych systemu.

2. Architektura Systemu i Zarządzanie Stanem

W celu zapewnienia maksymalnej wydajności, reaktywności oraz testowalności kodu, wdrożono zaawansowane zarządzanie stanem przy użyciu biblioteki `flutter_riverpod` (wersja 2.x+).

> **Zaleta Architektoniczna: Notifier i AsyncNotifier**
>
> Wdrożona struktura opiera się na nowoczesnych klasach `Notifier` i `AsyncNotifier`, które wyparły przestarzałe `StateNotifier`. Pozwala to na bezpieczne nasłuchiwanie, asynchroniczne ładowanie danych początkowych (stan `loading`/`data`/`error`) oraz zapewnia wbudowany dostęp do obiektu `ref` bezpośrednio z wnętrza metod mutujących stan (np. przy logowaniu lub dodawaniu aktywów).

3. Moduł Autentykacji i Ekran Logowania (LoginScreen)

3.1. Charakterystyka Funkcjonalna

Zaprojektowano i wdrożono główną „bramę wejściową” do aplikacji w postaci bezpiecznego, responsywnego ekranu logowania. Moduł został przygotowany w 100% zgodnie z wytycznymi backendowymi (Dev A) dotyczącymi obsługi tokenów JWT (JSON Web Token), a na obecnym etapie MVP funkcjonuje w oparciu o Mock API symulujące odpowiedzi serwera.

3.2. Komponenty Techniczne i Implementacja UI/UX

- **Nawigacja i Routing (Entry Point):** Zmodyfikowano rdzeń aplikacji (plik `main.dart`), ustawiając `LoginScreen` jako główny punkt startowy aplikacji (home). Po pomyślnej walidacji i uwierzytelnieniu, logika używa `Navigator.pushReplacement`, aby przenieść użytkownika do `MainNavigationShell`. Proces ten całkowicie zdejmuje ekran logowania ze stosu nawigacji, co zabezpiecza aplikację przed niepożądanym powrotem przyciskiem "wstecz".
- **Walidacja Formularza:** Dodano natywne, wbudowane zabezpieczenia dla kontrolek `TextFormField`. Adres e-mail jest sprawdzany pod kątem wymaganych znaków (np. @), a hasło musi zawierać bezpieczne minimum 6 znaków, zanim formularz zostanie przesłany do warstwy usługi.
- **Zarządzanie Stanem Sesji (AuthProvider):** Autentykacja opiera się na wydzielonym `NotifierProvider`, który poprzez usługę `ApiService` realizuje żądanie autoryzacji (simulated JWT Token fetch) i przechowuje flagę dostępności (isLoggedIn). Pozwala to w przyszłości na łatwe podpięcie mechanizmów typu `flutter_secure_storage` do trwałego zapisywania tokenu JWT na urządzeniu.
- **Elementy UX (Placeholdery):** Biorąc pod uwagę przyszłą rozbudowę oraz dobre praktyki UI/UX, dodano wizualne punkty zaczepienia (nieklikalne pola `onPressed: null`) dla przycisków resetowania hasła ("Zapomniałeś hasła?") oraz rejestracji ("Nie masz jeszcze konta? Zarejestruj się!").

4. Moduł Oszczędności (SavingsScreen)

4.1. Charakterystyka Funkcjonalna

Moduł umożliwia użytkownikowi kompleksowe zarządzanie poduszką finansową oraz rachunkami oszczędnościowymi. Widok prezentuje dynamicznie generowany wykres podziału aktywów oraz szczegółową listę kafelków reprezentujących poszczególne konta.

4.2. Komponenty Techniczne i Implementacja

- **Wykres Kołowy (Donut Chart):** Zaimplementowany przy użyciu pakietu `fl_chart`. Wykres w czasie rzeczywistym oblicza proporcje poszczególnych kont na podstawie zunifikowanej wartości bazowej.
- **Wielowalutowość i Normalizacja Danych:** Zgodnie z dobrymi praktykami inżynierii finansowej, wykres nie sumuje bezpośrednio nominalnych wartości różnych walut. Wdrożono wewnętrzny mechanizm walidacji i przeliczania walut obcych na PLN według bezpiecznych kursów (1 USD=4.00, 1 EUR=4.30, 1 GBP=5.05).

- **Formatowanie Prezentacji Liczb:** Napisano autorski parser (Regex) formatujący wyświetlane kwoty poprzez separację tysięcy przecinkami (np. `1,250,000 PLN`), co dramatycznie podnosi czytelność UI.

5. Moduł Inwestycji (InvestmentsScreen)

Moduł dedykowany do śledzenia rachunków maklerskich oraz kont emerytalnych. W ramach tego modułu zastosowano wzorzec **Reużywalności Kodu (Code Reuse)**.

Zamiast powielać formularz wprowadzania danych, uniwersalny ekran `AddAssetScreen` został wyposażony w flagi konstrukcyjne (`isInvestment`). Formularz automatycznie adaptuje swoje etykiety, nagłówki oraz kieruje strumień danych do właściwego zarządcy stanu (`investmentsProvider` zamiast `savingsProvider`).

6. Kalkulator Emerytalny (RetirementScreen)

Zgodnie z ustaleniami architektonicznymi dla MVP, kalkulator emerytalny w tej iteracji działa całkowicie po stronie klienta (Client-Side Computing), nie obciążając serwera zapytaniami sieciowymi. Zmiana parametrów na suwakach wywołuje natychmiastowe przeliczenie.

6.1. Logika Matematyczna (Silnik Obliczeniowy)

Silnik symulacji opiera się na wzorze finansowym na **wartość przyszłą renty z dołu (Future Value of an Ordinary Annuity)**, uwzględniającym regularne, comiesięczne wpłaty kapitału podlegające kapitalizacji miesięcznej:

$$FV = PMT \times ((1 + r)^n - 1) / r$$

Wdrożono również **Edge Case Protection**: Jeśli stopa zwrotu (`r`) zostanie ustawiona na 0%, algorytm automatycznie przełącza się na bezpieczne, liniowe mnożenie składek ($FV = PMT \times n$), zapobiegając błędowi dzielenia przez zero.

7. Integracja z Backendem i Zgodność z MVP

Interfejs frontendowy został precyzyjnie dostosowany do możliwości technicznych backendu określonych w dokumentacjach (Dev A i Dev B):

Operacja CRUD	Status na Serwerze	Implementacja we Flutterze (Dev D)
Create (Zapis)	Zaimplementowane (POST)	Pełna obsługa formularza, parametry wysyłane z walidacją
Read (Odczyt)	Zaimplementowane (GET)	Asynchroniczne listy i interaktywny wykres Donut Chart
Update (Edycja)	**Brak (Wyciszone w MVP)**	Kod UI wstrzymany (ołówek ukryty w widokach)
Delete (Usuwanie)	Zaimplementowane (DELETE)	Działająca ikonka kosza sprzężona z bazą danych (Kaskadowe)

8. Repozytorium i Przepływ Pracy (Git Workflow)

Prace deweloperskie były prowadzone zgodnie z dobrymi praktykami GitFlow. Poszczególne pakiety zmian były oddzielone na dedykowanej gałęzi `feature/add-asset-form`, a następnie włączone (merged) do głównej osi aplikacji:

```
$ git status
$ git add .
$git commit -m "Feat: Zakończenie prac nad MVP dla Dev D (Inwestycje, Kalkulator,
Wykres)"$ git commit -m "Feat: Dodanie ekranu logowania (Mock JWT), aktualizacja
AuthProvidera do Riverpod 2.0 i ustawienie punktu wejściowego w main.dart"
$ git checkout main
$git merge feature/add-asset-form$ git push origin main
```

[Koniec Dokumentu - Wersja Cyfrowa PDF]